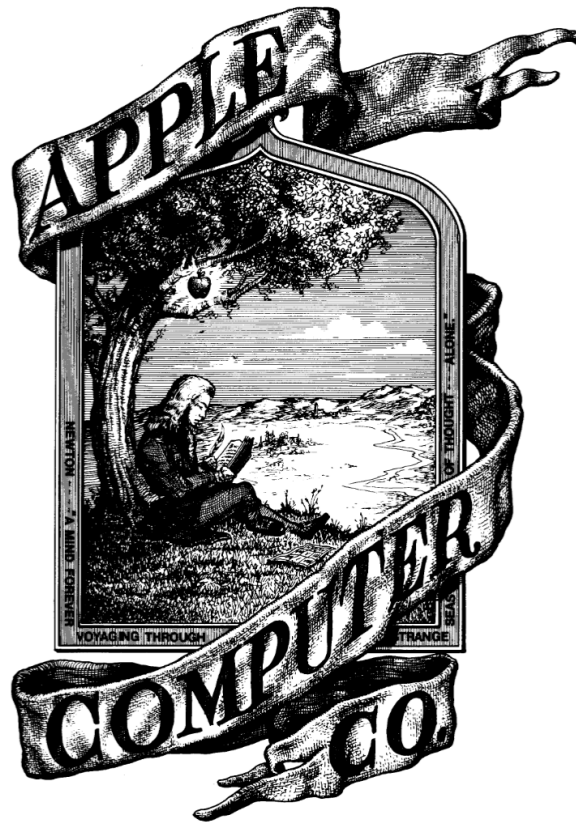




APPLE-1

OPERATION MANUAL

APPLE COMPUTER COMPANY

770 Welch Road

Palo Alto, California 94304

SPECIFICATIONS

MICROPROCESSOR:	MOS Technology 6502
Microprocessor Clock Frequency:	1.023 MHz
Effective Cycle Frequency: (Including Refresh Waits)	0.960 Mhz
VIDEO OUTPUT:	Composite positive video, 75 ohms, level adjustable between zero and +5V pp.
Line Rate:	15734 Hz
Frame Rate:	60.06 Hz
Format	40 characters/line, 24 lines; with automatic scrolling
Display Memory:	Dynamic shift registers (1K x 7)
Character Matrix:	5 x 7

INTRODUCTION

The Apple Computer is a complete microprocessor system, consisting of a Mos Technology 6502 microprocessor and support hardware, integral video display electronics, dynamic memory and refresh hardware, and fully regulated power supplies. It contains resident system monitor software, enabling the user, via the keyboard and display, to write, examine, debug, and run programs efficiently; thus being an educational tool for the learning of microprocessor programming, and an aid in the development of software.

The integral video display section and the keyboard interface renders unnecessary the need for an external teletype. The display section contains its own memory, leaving all of RAM for user programs, and the output format is 40 characters/line, 24 lines/page, with auto scrolling. Almost any ASCII encoded keyboard will interface directly with the Apple system.

The board has sockets for upto 8K bytes of the 16 pin, 4K type, RAM, and the system is fully expandable to 65K via the edge connector. The system uses dynamic memory (4K bytes supplied), although static memory may

also be used. All refreshing of dynamic memory, including all "off-board" expansion memory, is done automatically. The entire system timing, including the microprocessor clock and all video signals, originates in a single crystal oscillator.

Further, the printed circuit board contains a "breadboard area", in which the user can add additional "on-board" hardware (for example, extra PIA's, ACIA's, EROM's, and so on).

This manual is divided into three Sections:

Section I GETTING THE SYSTEM RUNNING

Section II USING THE SYSTEM MONITOR. (listing included)

Section III EXPANDING THE SYSTEM.

Please read Section I thoroughly, before attempting to "power-up" your system, and study Section III carefully before attempting to expand your system. In addition to this manual, Apple 'Tech Notes' are available which contain examples of expansion hardware and techniques.

SECTION I GETTING THE SYSTEM RUNNING

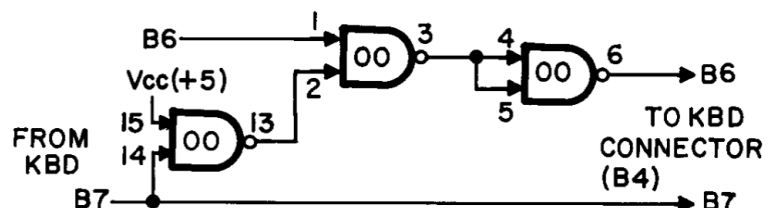
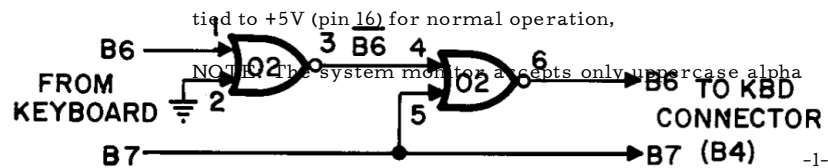
The Apple Computer is fully assembled, tested, and burned in. The only external devices necessary for operation of the system are: An ASCII encoded keyboard, a video display monitor, and AC power sources of 8 to 10 Volts (RMS) @3 amps and 28Volts(RMS)@1 amp. The following three articles describe the attachment of these devices in detail.

Keyboard:

Any ASCII encoded keyboard, with positive DATA outputs, interfaces directly with the Apple system via a "DIP" connector. If your keyboard has negative logic DATA outputs (rare), you can install inverters (7404) in the breadboard area. The strobe can be either positive or negative, of long or short duration. The 'DIP' keyboard connector (B4) has inputs for seven DATA lines, one STROBE line, and two normally-open pushbutton switches, used for RESET (enter monitor), and CLEAR SCREEN (see schematic diagram, sheet 3 of 3, for exact circuitry). This keyboard connector also supplies three voltages, (+5V, +12V, and -12V) of which one or more may be necessary to operate the keyboard. Pin 15 of the keyboard connector (B4) must be tied to +5V (pin 16) for normal operation,

(A-F, R).

It is therefore convenient, though it's not essential, to have a keyboard equipped with uppercase alpha lock (usually in the electronics). Either of the following suggested circuits may be used to provide alpha lock capability, if needed, and can be built in the breadboard area.



Display:

The Apple Computer outputs a composite video signal (composite of sync and video information) which can be applied to any standard raster-scan type video display monitor. The output level is adjustable with the potentiometer located near the video output Molex connector, J2. The additional two outside pins on the Molex connector supply +5 and +12 volts, to be used in future Apple accessories. The composite video signal can also be modulated at the proper RF frequency, with an inexpensive commercially available device, and applied to the antenna terminals of a home television receiver. Since the character format is 40 characters /line, all television receivers will have the necessary bandwidth to display the entire 40 characters. Two large manufacturers of video display monitors, which connect directly with the Apple Computer, are Motorola and Ball. The mating four-pin Molex connector is provided.

AC Power Sources:

Two incoming AC power sources are required for operation: 8 to 10 VAC (RMS) at 3 amps, and 28 VAC (RMS) Center-Tapped at 1 amp. These AC supplies enter the system at the Molex connector, J1. The 8 to 10 volts AC provides the raw AC for the +5 volt supply, while the 28 VCT supplies the raw AC for the +12 and -12 volt supplies, and the -5V supply is derived from the -12V regulated output.

The board, as supplied, requires no more than 1.5 amps DC from the 45V supply, while the regulator is capable of supplying 3 amps. The remaining 1.5 amps DC from the 45V supply is available for user hardware expansion (provided suitable transformer ratings are employed).

A suitable source of the raw AC voltages required, are two commercially available transformers; Stancor P/N P-8380 or equivalent (8 to 10 volts at 3 amps), and Stancor P/N P-8667 or

equivalent (28VCT at 1 amp). Simply wire the secondaries to the mating six pin Molex connector supplied, and

wire the primaries in parallel, as shown in the schematic diagram (power supply section, Dwg.No. 00101, sheet 3 of 3).

TEST PROGRAM

After attaching the keyboard, display, and AC power sources, you can try a simple program to test if your system and the attachments are functioning together properly. While it does not test many possible areas of the microprocessor system, the test program will test for the correct attachment of the keyboard, display, and power supplies.

FIRST:

Hit the RESET button to enter the system monitor. A backslash should be displayed, and the cursor should drop to the next line,

SECOND:

Type- `0:A9b0 bAAK20bEFbDFF bd E8 b 8A b4C b2b0 (RET)`

(0 is a zero, NOT an alpha "O"; b means blank or space; and (RET) hit the "return" key on the keyboard)

THIRD:

Type- `0.A(RET)`

(This should print out, on the display, the program you have just entered,)

FOURTH: Type- `R(RET)` (R means run the program.)

THE PROGRAM SHOULD THEN PRINT OUT ON THE DISPLAY A CONTINUOUS STREAM OF ASCII CHARACTERS, TO STOP THE PROGRAM AND RETURN TO THE SYSTEM MONITOR, HIT THE "RESET" BUTTON. TO RUN AGAIN, TYPE: `R (RET)`.

SECTION II USING THE SYSTEM MONITOR

The Hex Monitor is a PROM program in locations FF00 to FFFF (hex) which uses the keyboard and display to perform the front panel functions of examining memory, and running programs. The monitor program is entered by hitting (RESET), which displays backslash-return. A backslash alone (cursor remains on same line as backslash) indicates bad page 0 RAM.

Commands are typed on a "line-at-a-time" basis with editing. Each line may consist of any number of commands (up to 128 characters). None are executed until (RETURN) is typed. The (SHIFT-0) (backarrow) backspaces and echos an underline. The (ESC) cancels a line and echos backslash-return.

One or more hexadecimal digits (0-9, A-F) are used for address and data values. Addresses use the four least significant digits of a group, and data values, the two least significant digits. The following examples illustrate the variety of acceptable commands:

1. Opening a location (examining the contents of a single address).

USER TYPES/ 4F (RET)

MONITOR TYPES/ 004F: 0F (contents of 4F)

2. Examining a block; from the last examined location, to a specified one.

USER TYPES/ .5A (RET)

MONITOR TYPES/ 0050: 00 01 02 03 04 05 06 07
0058: 08 09 0A

Note: 4F is still considered the most recently opened location.

3. Combining examples 1 and 2 to print a block of memory in a single command.

USER TYPES/ 4F,5A (RET)

MONITOR TYPES/ 0050: 00 01 02 03 04 05 06 07
0058: 08 09 0A

Note: Only the first location of the block (4F) is considered "opened!"

4. Examining several individual locations at once.

USER TYPES/ AF b 52 b 56 (RET)

MONITOR TYPES/ 004F: 0F 0052: 02 0056: 06

Note:

56 is considered the most recently "opened" location. The 'b' is a blank or comma, and is a delimiter for separation purposes only. A string of delimiters has the same effect as a single one (bbb is as effective as b).

5. Examining several blocks of memory at once.

USER TYPES/ 4F. 52 b 56 b 58.5A (RET)

MONITOR TYPES/ 004F: 0F

0052: 02

0056: 06

0058: 08 09 05A: 0A

Note: 58 is considered the most recently opened" location. Refer to example 2.

6. Examining successive blocks.

USER TYPES/ 4F.52 (RET)

MONITOR TYPES/ 004F: 0F 0050: 00 01 02

USER TYPES/ .55 (RET)

MONITOR TYPES/ 0053: 03 04 05

USER TYPES/ .5A (RET)

MONITOR TYPES/ 0056: 06 07

0058: 08 09 0A

7. Depositing data in a single location.

USER TYPES/ 30: A0 (RET)

MONITOR TYPES/ 0030: FF (prior contents)

Location 39 is considered opened and now contains 39.

8. Depositing data in successive locations from that last used in a deposit command.

USER TYPES/ : A1 b A2 b A3 b A4

b A5 (RET)

(This deposits A1 in location 31, A2 in 32, and so on.)

9. Combining examples 7 and 8 in a single command.

USER TYPES/ 30: A0 b A1 b A2 b

A3 b A4 b A5 (RET)

MONITOR TYPES/

0030: FF (prior contents of location 30)

10. Depositing data in successive locations with separate commands.

USER TYPES/ 30: A0 b A1 (RET) MONITOR TYPES/
0030: FF USER TYPES/ :A2 b A3 (RET) USER
TYPES/ :A4 b A5 (RET)

Note: A colon in a command means "start depositing data from the most recently deposited location, or if none, then from the most recently opened one.

11. Examining a block, then depositing into it.

USER TYPES/ 30.35 (RET)

MONITOR TYPES/

0030: A0 A1 A2 A3 A4 A5 A6

USER TYPES/

:B0 b B1 b B2 b B3 b B4 b B5 (RET)

Note: New data deposited beginning at most recently opened location (39)

12. Run a program at a specified address.

USER TYPES/ 10F0 R (RET)

MONITOR TYPES/ 10F0: A9 (contents)

Note: The cursor is left immediately to the right of the 'A9'; it is not returned to the next line.

13. Run at the most recently examined location.

USER TYPES/ 10F0 (RET)

MONITOR TYPES/ 10F0: A9

USER TYPES/ R (RET)

14. Enter a program into memory and run it in one line.

USER TYPES/ 40: A9 b 0 b 20 b EF b FF b 38 b 69 b

MONITOR TYPES/ 40: FF (prior contents of 40)

MONITOR TYPES/ 40: FF (prior contents of 40)

15. An "on line" error correction.

USER TYPES/

40: A1 b A2 b A3A4A5A6 b A7

(data A6 will be loaded in location 42)

USER TYPES/ 40506070: AA

(data AA will be loaded in location 6070)

16. Useful routines in monitor which can be accessed by user programs.

GETLINE: location FF1F:

monitor entry point

(jumping to FF1F will enter monitor and echo carriage return. You can then examine memory locations with the monitor.)

ECHO: location FFEF:

prints one byte (ASCII)

(data from "A" (accumulator), contents of "A" not disturbed. Example:

20 b EF b FF (JRS ECHO)).

PRBYTE: location FFDC:

prints one byte (HEX)

(data from "A", contents of "A" disturbed.)

PRHEX: location FFES5:

prints one hex digit

(data from four least significant bits of "A", contents of "A" disturbed.)

NOTE: Capital letters enclosed in parenthesis represent single keystrokes. Example: (RET) means hit the "return" key

NOTE: RAM locations 0024 to 002B are used as index pointers by the monitor, and are invalid for user use, when using monitor. Also, locations 0020 to 027F are used as input buffer storage, and are also invalid for user use when using the monitor.

6502 HEX MONITOR LISTING

FF00	D8	RESET	CLD	Clear Decimal arithmetic mode
FF01	58		CLI	
FF02	A0 7F		LDY #\$7F	Mask for DSP data direction register
FF04	8C 12 D0		STY DSP	Set it up
FF07	A9 A7		LDA #\$A7	KBD and DSP control register mask
FF09	8D 11 D0		STA KBD CR	Enable interrupts, set CA1, CBI, for
FF0C	8D 11 D0		STA DSP CR	positive edge sense/output mode
FF0F	8D 13 D0	NOTOR	CMP #\$DF	"<="?
FF11	F0 13		BEQ BACKSPACE	Yes.
FF13	C9 9B		CMP #\$9B	ESC?
FF15	F0 03		BEQ ESCAPE	Yes.
FF17	C8		INY	Advance text index.
FF18	10 0F		BPL NEXTCHAR	Auto ESC if · 127.
FF1A	A9 DC	ESCAPE	LDA #\$DC	"\".
FF1C	20 EF FF		JSR ECHO	Output it.
FF1F	A9 8D	GETLINE	LDY #\$01	CR.
FF21	20 EF FF		JSR ECHO	Output it.
FF24	A0 01		LDY #\$01	Initialize text index.
FF26	88	BACKSPACE	DEY	Back up text index.
FF27	30 F6		BMI GETLINE	Beyond start of line, reinitialize.
FF29	AD 11 D0	NEXTCHAR	LDA KBD CR	Key ready?
FF2C	10 FB		BPL NEXTCHAR	Loop until ready.
FF2E	AD 10 D0		LDA KBD	Load Character. B7 should be 'I'.
FF31	99 00 02		STA IN, Y	Add to text buffer.
FF34	20 EF FF		JSR ECHO	Display character.
FF37	C9 8D		CMP #\$8D	CR?
FF39	D0 D4		BNE NOTOR	No.
FF3B	A0 FF		LDY #\$FF	Reset text index.
FF3D	A0 FF		LDA #\$00	For XAM mode.
FF3F	AA		TAX	0 → X
FR40	0A	SETSTOR	ASL	Leaves \$7B if setting STOR mode.
FF41	85 2B	SETMODE	STA MODE	\$00 = XAM, \$7B = STOR, \$AE = BLOK XAM.
FF43	C8	BLSKIP	INY	Advance text index.
FR44	B9 00 02	NEXT ITEM	LDA IN, Y	Get character.
FF47	C9 8D		CMP #\$8D	CR?
FF49	F0 D		BEQ GETLINE	Yes, done this line.
FF4B	C9 AE		CMP #\$AE	".,?"
FF4D	90 F4		BCC BLSKIP	Skip delimiter.
FF4F	F0 F0		BEQ SETMODE	Set BLOCK XAM mode.
FF51	C9 BA		CMP #\$BA	".,?"
FF53	F0 EB		BEQ SETSTOR	Yes, set STOR mode.
FF55	C9 D2		CMP #\$D2	"R"?
FF57	F0 3B		BEQ RUN	Yes, run user program.
FF59	86 28		STX L	\$00 → L.
FF5B	86 29		STX H	and H.
FF5D	84 2A		STY YSAV	Save Y for comparison.
FF5F	B9 00 02	NEXTHEX	LDA IN, Y	Get character for hex test.
FF62	49 B0		EOR #\$B0	Map digits to \$0-9.
FF64	C9 00		CMP #\$0A	Digit?
FF66	90 06		BCC DIG	Yes.
FF68	69 88		ADC #\$88	Map letter "A"- "F" to \$FA-FF.
FF6A	C9 FA		CMP #\$FA	Hex letter?

FF6C	90 11		BCC NOTHEX	No, character not hex.
FF6E	0A	DIG	ASL	
FF6F	0A		ASL	Hex digit of MSD of A.
FF70	0A		ASL	
FF71	0A		ASL	
FF72	A2 04		LDX #04	Shift count.
FF74	0A	HEXSHIFT	ASL	Hex digit left, MSB to carry
FF75	26 28		ROL L	Rotate into LSD.
FF77	26 29		ROL H	Rotate into MSD's.
FF79	CA		DEX	Done 4 shifts?
FF7A	D0 F8		BNE HEXSHIFT	No, loop.
FF7C	C8		INY	Advance text index.
FF7D	D0 E0		BNE NEXTHEX	Always taken. Check next character for hex.
FF7F	C4 2A	NOTHEX	CPY YSAV	Check if L, H empty (no hex digits).
FF81	F0 97		BEQ ESCAPE	Tes, generate ESC sequence.
FF83	24 2B		BITMODE	Test MODE byte.
FF85	50 10		BVC NOTSTOR	B6 = 0 for STOR, 1 for XAM and BLOCK XAM
FF87	A5 28		LDA L	LSD's of hex data.
FF89	81 26		STA (STL, X)	Store at current 'store index'.
FF8B	E6 26		INC STL	Increment Store index.
FF8D	D0 B5		BNE NEXTITEM	Get next item (no carry).
FF8F	E6 27		INC STH	Add carry to 'store index' high order.
FF91	4C 44 FF	TONEXTITEM	JMP NEXTITEM	Get next command item.
FF94	6C 24 00	RUN	JMP (XAML)	Run at current XAM index.
FF97	30 2B	NOTSTOR	BMI XAMNEXT	B7 = 0 for XAM, 1 for BLOCK XAM.
FF99	A2 02		LDX #02	Byte count.
FF9B	B5 27	SETADR	LDA L-1, X	Copy hex data to
FF9D	95 25		STA ST; -1, X	'store index'.
FF9F	95 23		STA XAML-1, X	And to 'XAM index'
FFA1	CA		DEX	Next 2 bytes.
FFA2	D0 F7		BNE SETADR	Loop unless X = 0.
FFA4	D0 14	NXTPRNT	BNE PRDATA	NE means no address to print.
FFA6	A9 8D		LDA #8D	CR.
FFA8	20 EF FF		JSR ECHO	Output it.
FFAB	A5 25		LDA XAMH	'Examine index' high-order byte.
FFAD	20 DC FF		JSR PRBYTE	Output it in hex format.
FFB0	A5 24		LDA XAML	Low-order 'examine index' byte.
FFB2	20 DC FF		JSR PRBYTE	Output it in hex format.
FFB5	A9 BA		LDA #BA	".:".
FFB7	20 EF FF		JSR ECHO	Output it.
FFBA	A5 24	PRDATA	LDA #A0	Blank.
FFBC	20 EF FF		JSR ECHO	Output it.
FFBF	A1 24		LDA (XAML, X)	Get data byte at 'examine index'.
FFC1	20 DC FF		JSR PRBYTE	Output it in hex format.
FFC4	86 2B	XAMNEXT	STX MODE	0 → MODE (XAM mode).
FFC7	A5 24		LDA XAML	
FFC8	C5 28		CMP L	Compare 'examine index' to hex data.
FFCA	A5 25		LDA XAMH	
FFCC	E5 29		SBC H	
FFCE	B0 C1		BCS TONEXTITEM	Not less, so no more data to output.
FFD0	E6 24		INC XAML	
FFD2	D0 02		BNE MOD8CHK	Increment 'examine index'.
FFD4	E6 25		INC XAMH	
FFD6	A5 24	MOD8CHK	LDA XAML	Check low-order 'examine index' byte

FFD8	29 07		AND #\$07	For MOD 8= 0
FFDA	10 C8		BPL NXTPRNT	Always taken.
FFDC	48	PRBYTE	PHA	Save A for LSD.
FFDD	4AA		LSR	
FFDE	4A		LSR	
FFDF	4A		LSR	MSD to LSD position.
FFE0	4A		LSR	
FFE1	20 E5 FF		JSR PRHEX	Output hex digit.
FFE4	68		PLA	Restore A
FFE5	29 0F	PRHEX	AND #\$0F	Mask LSD for hex print
FFE7	09 B0		ORA #\$B0	add "0".
FFE9	C9 BA		CMP #\$BA	Digit?
FFEB	90 02		BCC ECHO	Yes, output it.
FFED	69 06		ADC #\$06	Add offset for letter.
FFEF	2C 12 D0	ECHO	BIT DSP	DA bit (B7) cleared yet?
FFF2	30 FB		BMI ECHO	No, wait for display.
FFF4	8D 12 D0		STA DSP	Ooutput character. Sets DA.
FFF7	60		RTS	Return
FFF8	00 00 (unused)			
FFFA	00 0F (NMI)			
FFFC	00 FF (RESET)			
FFFE	00 00 (IRQ)			